# RECURRENT NEURAL NETWORKS
## & SEQUENCE MODELING

*Complete Study Notes | Exam Ready*

### 🎯 WHAT YOU WILL LEARN

- Sequence models and how RNNs process time-series/text/speech
- RNN architecture, equations, and parameter counting
- Vanishing/Exploding gradients and how to fix them
- LSTM & GRU — gates, equations, working
- Bidirectional RNNs, Deep RNNs, Attention Mechanism
- Real-world applications: NLP, speech, time-series

# UNIT 1: FUNDAMENTALS OF SEQUENCE MODELS

## What is Sequential Data?

Unlike tabular data, **sequential data has ORDER that matters**. The value at time step **t** depends on previous values.

**📌 Key Concept: Sequential Data**

**Types of Sequential Data:**

- **Time-Series:** Stock prices, sensor readings, weather temperature over days
- **Text (NLP):** Words in a sentence: "The cat sat on the mat" — order is critical
- **Speech:** Audio waveforms sampled at fixed intervals (e.g., 16000 Hz)

## Key Concepts

- **Dependency Across Time Steps:** In "I am going to France... I speak French", "French" depends on "France" earlier.
- **Hidden State (Memory):** A vector that carries information from past time steps forward into the future.

## RNN Architecture Types

An RNN can be configured to map inputs/outputs in 4 different ways:

| Architecture | Input | Output | Example |
|---|---|---|---|
| One-to-One | Single x | Single y | Image classification |
| One-to-Many | Single x | Sequence y | Image captioning |
| Many-to-One | Sequence x | Single y | Sentiment analysis |
| Many-to-Many | Sequence x | Sequence y | Machine translation |

**💡 Example: Many-to-Many (Machine Translation)**
Input: "Je suis étudiant" (3 tokens) → Output: "I am a student" (4 tokens)
Input sequence and output sequence can have DIFFERENT lengths!

## Working: How RNN Processes Sequences

**⚙️ Step-by-Step RNN Processing:**

- **Step 1:** At time step t=1, take input $x_1$ and initial hidden state $h_0 = 0$
- **Step 2:** Compute new hidden state: $h_1 = f(W \cdot x_1 + U \cdot h_0 + b)$
- **Step 3:** (Optionally) produce output $y_1 = g(V \cdot h_1)$
- **Step 4:** Pass $h_1$ as input to next time step t=2
- **Repeat:** Until all T time steps are processed

## 📐 Numerical/Exam Focus: Shapes

| Parameter | Shape / Details |
|---|---|
| Input x_t | (input_size, 1) per time step |
| Hidden state h_t | (hidden_size, 1) |
| Output y_t | (output_size, 1) |
| Weight W_xh | (hidden_size, input_size) |
| Weight W_hh | (hidden_size, hidden_size) |
| Weight W_hy | (output_size, hidden_size) |

# UNIT 2: BASIC RNN ARCHITECTURE

## Core Equations

🔢 **Hidden State Update Equation:**

$$h_t = \tanh(W_{hh} \cdot h_{(t-1)} + W_{xh} \cdot x_t + b_h)$$

🔢 **Output Equation:**

$$y_t = W_{hy} \cdot h_t + b_y$$

## Important Terms Explained

| Symbol / Term | Meaning |
|---|---|
| h_t | Hidden state at time t — the 'memory' of the network |
| x_t | Input vector at time step t |
| W_hh | Weight matrix: hidden-to-hidden connections (recurrent weights) |
| W_xh | Weight matrix: input-to-hidden connections |
| W_hy | Weight matrix: hidden-to-output connections |
| b_h, b_y | Bias vectors for hidden and output layers |
| tanh | Activation function — outputs values between -1 and +1 |

## Key Property: Weight Sharing

- **Same weights (W_hh, W_xh, W_hy) are REUSED at every time step** — this is what makes RNNs efficient!
- This is called **parameter sharing** and reduces the total number of parameters
- But it also causes the **vanishing gradient problem**

## 📐 Numerical: Parameter Count

Given: input_size = n, hidden_size = h, output_size = m

**🧮 Parameter Count Formula**

- `W_xh:` h × n parameters
- `W_hh:` h × h parameters
- `b_h:` h parameters
- `W_hy:` m × h parameters
- `b_y:` m parameters

**Total = h×n + h×h + h + m×h + m**

**Example:** n=10, h=5, m=3 → 5×10 + 5×5 + 5 + 3×5 + 3 = 50+25+5+15+3 = 98 parameters

# UNIT 3: VANISHING & EXPLODING GRADIENT PROBLEM

## The Core Problem

During **Backpropagation Through Time (BPTT)**, gradients are multiplied through many time steps. This causes two critical problems:

| ⚠️ VANISHING GRADIENTS | 💥 EXPLODING GRADIENTS |
|---|---|
| **Cause:** Gradient < 1, multiplied repeatedly → approaches 0<br>**Effect:** Network forgets long-term dependencies<br>**Training:** Loss stops improving, weights barely change | **Cause:** Gradient > 1, multiplied repeatedly → very large<br>**Effect:** Weight updates become huge, unstable training<br>**Training:** Loss oscillates wildly, NaN values appear |

## Mathematical Cause

Gradient at time step t=1 through T time steps:

$$\partial L/\partial h_1 = (\partial h_T/\partial h_1) \cdot (\partial L/\partial h_T) = \Pi(\partial h_k/\partial h_{k-1}) \cdot \ldots$$

If **||W_hh|| < 1** → product shrinks → **Vanishing**
If **||W_hh|| > 1** → product grows → **Exploding**

## Solutions

### ✅ Solutions to Gradient Problems

- **Gradient Clipping:** If ||gradient|| > threshold, scale it down. Prevents explosion.
  - `if ||g|| > max_norm: g = g * max_norm / ||g||`
- **Proper Initialization:** Xavier/Glorot init helps gradients flow evenly at start
- **Use LSTM/GRU:** Specially designed to maintain long-term memory without vanishing gradients
- **Skip Connections / Residual:** Provide gradient highways through the network

# UNIT 4: BACKPROPAGATION THROUGH TIME (BPTT)

## Concept

BPTT is the algorithm used to train RNNs. It is simply **backpropagation applied on an UNROLLED RNN**, treating each time step as a layer in a standard neural network.

⚙️ **How BPTT Works:**

- **1. Forward Pass:** Run RNN for all T time steps, store all h_t and y_t values
- **2. Compute Loss:** L = Σ L_t (sum of losses at each time step)
- **3. Backward Pass:** Unroll the RNN and apply chain rule backward through all T steps
- **4. Sum Gradients:** Since weights are shared, sum gradients across all time steps
- **5. Update Weights:** W = W - α · ΣΔW (gradient descent update)

## Full vs Truncated BPTT

| Type | Description |
|---|---|
| Full BPTT | Backpropagate through ALL T time steps. Accurate but expensive for long sequences. |
| Truncated BPTT | Backpropagate through only k steps. Faster and avoids extreme vanishing gradients. Common in practice. |

## Chain Rule Application (Exam Focus)

**The key chain rule equation:**

$$\partial L/\partial W = \Sigma(t=1 \text{ to } T)\ \partial L_t/\partial W$$

Complexity: O(T) forward + O(T) backward → O(T) total per sequence

# UNIT 5: LONG SHORT-TERM MEMORY (LSTM)

## Why LSTM?

Standard RNNs **FAIL to remember long-term dependencies** due to vanishing gradients. LSTM solves this with a **cell state (C_t)** — a dedicated memory highway that carries information across many time steps with minimal degradation.

## LSTM Architecture Overview

| Component | Role |
|---|---|
| **Cell State (C_t)** | The 'long-term memory' — information can flow unchanged across many steps |
| **Hidden State (h_t)** | The 'short-term memory' / output — filtered version of cell state |
| **Forget Gate (f_t)** | Controls what percentage of previous cell state to KEEP (0=forget all, 1=keep all) |
| **Input Gate (i_t)** | Controls what NEW information to WRITE into cell state |
| **Output Gate (o_t)** | Controls what portion of cell state to OUTPUT as hidden state |

## LSTM Gate Equations

| Gate | Symbol | Formula | Purpose |
|---|---|---|---|
| **Forget Gate** | f_t | $\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ | Decides what to REMOVE from cell state |
| **Input Gate** | i_t | $\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ | Decides what NEW info to ADD |
| **Output Gate** | o_t | $\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ | Decides what to OUTPUT from cell |
| **Cell Candidate** | C̃_t | $\tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$ | New candidate values for cell state |

## Cell & Hidden State Updates

**Cell state update:**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

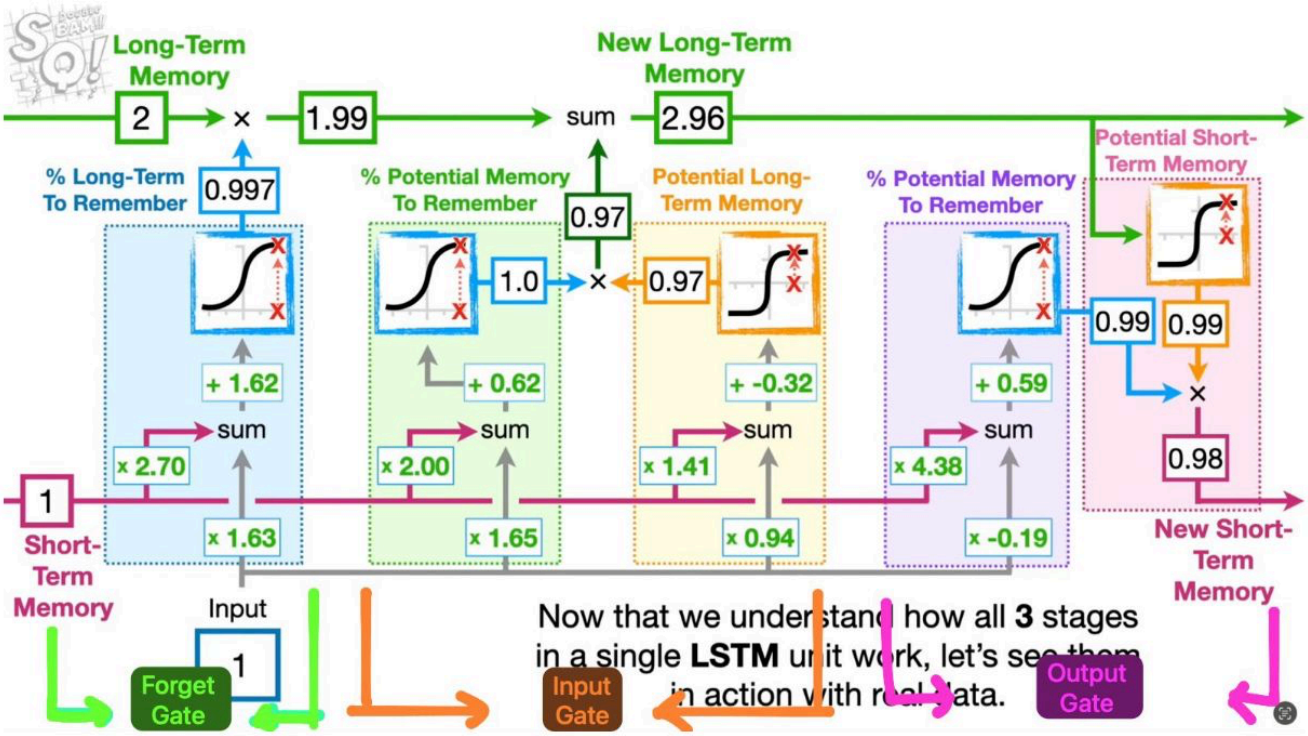
$f_t \odot C_{t-1}$ = what to keep from old memory; $i_t \odot \tilde{C}_t$ = new info to add

**Hidden state output:**

$$h_t = o_t \odot \tanh(C_t)$$

💡 **Memory Aid — LSTM as a Filing Cabinet:**

- **Forget Gate:** Like a SHREDDER — "What old files should I throw away?"
- **Input Gate:** Like a PRINTER — "What new documents should I file?"
- **Output Gate:** Like a PHOTOCOPIER — "What should I take out to use?"



# UNIT 6: GATED RECURRENT UNIT (GRU)

## GRU — Simplified LSTM

GRU was proposed in 2014 as a **simpler and faster alternative to LSTM**. It merges the forget and input gates into a single **Update Gate** and adds a **Reset Gate**. No separate cell state.

## GRU Equations

**Update Gate (z_t):** How much of past to carry forward vs. new info

```
z_t = σ(W_z · [h_{t-1}, x_t])
```

**Reset Gate (r_t):** How much of past hidden state to forget

```
r_t = σ(W_r · [h_{t-1}, x_t])
```

**Candidate Hidden State (h̃_t):**

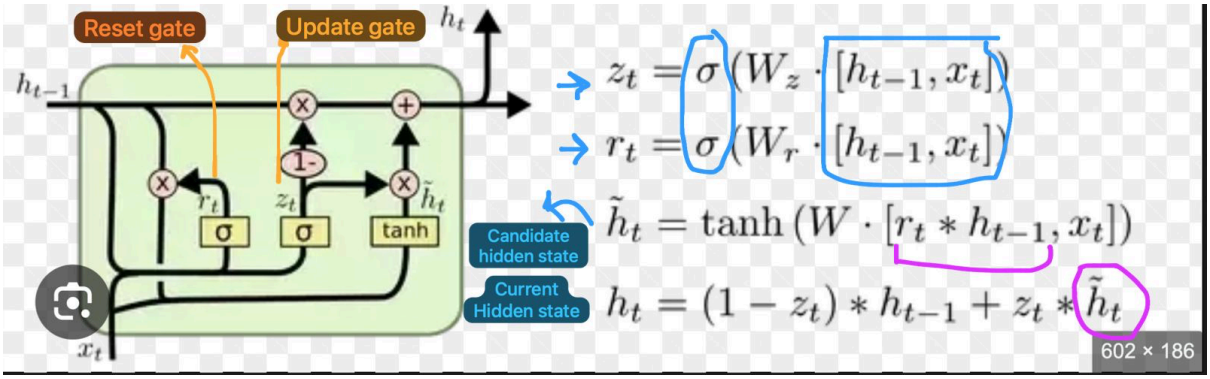
```
h~_t = tanh(W · [r_t ⊙ h_{t-1}, x_t])
```

**Final Hidden State:**

```
h_t = (1 - z_t) ⊙ h_{t-1} + z_t ⊙ h~_t
```

## GRU vs LSTM Comparison

| Feature | LSTM | GRU |
|---|---|---|
| Gates | 3 gates (Forget, Input, Output) | 2 gates (Update, Reset) |
| Cell State | Separate C_t and h_t | Only h_t (no separate cell) |
| Parameters | More parameters (4 weight matrices) | Fewer parameters (3 weight matrices) |
| Speed | Slower to train | Faster to train |
| Performance | Better for very long sequences | Comparable on most tasks |
| Complexity | More complex architecture | Simpler architecture |



# UNIT 7: BIDIRECTIONAL RNN (BiRNN)

## Concept & Motivation

Standard RNN only sees **PAST context (left to right)**. But in many tasks, **FUTURE context also matters!**

**Example:** "I will [MASK] you tomorrow" → The word before AND after "MASK" matters for prediction.

## How BiRNN Works

### ⚙️ Architecture

- **Forward RNN:** Processes sequence left → right: h_t_fwd captures past context
- **Backward RNN:** Processes sequence right → left: h_t_bwd captures future context
- **Output:** Concatenate both: ŷ_t = concat(h_t_fwd, h_t_bwd)

- **Output size = 2 × hidden_size** (due to concatenation)

## Use Cases & Limitations

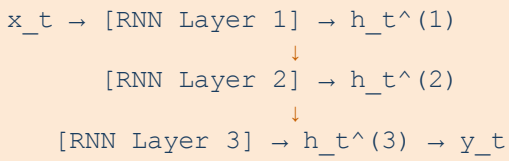
| Aspect | Details |
|---|---|
| Good For | Named Entity Recognition, POS tagging, Speech Recognition (offline) |
| NOT for | Real-time generation tasks (can't look at future for live prediction) |
| Output Dim | 2 × hidden_size (forward + backward concatenated) |
| Parameters | 2× a standard RNN (two independent RNNs) |

# UNIT 8: DEEP RNNs (STACKED RNNs)

## Concept

A **Deep RNN stacks multiple RNN layers**. The output (hidden state) of Layer 1 becomes the input to Layer 2, and so on.

**Flowchart: 3-Layer Deep RNN**




## Advantages & Exam Points

- **Better Representations:** Lower layers learn basic patterns, higher layers learn abstractions
- **Dimension:** Output of Layer l has dimension = hidden_size_l (can differ per layer)
- **Parameter Count:** Multiply per layer; each layer adds its own W_xh, W_hh, W_hy
- **Typical depth:** 2-4 layers for RNNs (deeper often uses residual connections)

# UNIT 9: OPTIMIZATION FOR LONG-TERM DEPENDENCIES

## Why is it Hard?

As sequences get longer, the **gradient signal weakens (vanishes)** or **explodes**. RNNs struggle with dependencies spanning 10+ time steps.

## Optimization Techniques

### ✅ Techniques

- **LSTM / GRU:** Primary solution — gated architectures designed for long-term memory
- **Gradient Clipping:** Cap gradient norm to prevent explosion. Typically max_norm = 1.0 or 5.0
- **Better Activations:** ReLU can help reduce vanishing (no saturation zone), but can cause explosion
- **Batch Normalization:** Normalize activations — less widely used in RNNs than CNNs
- **Layer Normalization:** More common in RNNs — normalizes per-sample per-timestep
- **Attention Mechanism:** Allows direct connections between any two positions — bypasses gradient bottleneck

### 📝 Exam Question: Why does LSTM work better than RNN?

**Answer:** LSTM has an explicit cell state with additive updates ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$). The **additive update prevents gradient from shrinking multiplicatively** through time. Gates selectively protect important information from being overwritten, solving the vanishing gradient problem.

# UNIT 10: ATTENTION MECHANISM IN SEQUENCES

## Motivation

In encoder-decoder models for translation, the encoder compresses the ENTIRE input into a **single fixed-size vector**. This bottleneck loses information for long sequences. **Attention allows the decoder to LOOK BACK at all encoder outputs!**

## How Attention Works

### ⚙️ Attention Steps

- **Step 1 — Score:** Compute alignment score between decoder state s and each encoder output h_i
  - `e_i = score(s, h_i)   e.g., e_i = v^T · tanh(W_s·s + W_h·h_i)`
- **Step 2 — Normalize (Softmax):** Convert scores to probabilities (attention weights)
  - `α_i = softmax(e_i) = exp(e_i) / Σ exp(e_j)`
- **Step 3 — Context Vector:** Weighted sum of encoder outputs
  - `c = Σ α_i · h_i`
- **Step 4 — Decode:** Use context vector c alongside decoder state to produce output

## Types of Attention

| Type | Description |
|---|---|
| Bahdanau (Additive) | Uses a feedforward network to compute scores. Original attention (2015). |
| Luong (Dot-Product) | Uses dot product between decoder and encoder states. Simpler and faster. |
| Self-Attention | Each position attends to ALL other positions in the same sequence. Used in Transformers. |

## Benefits of Attention

- **Handles long sequences much better** — no single bottleneck vector
- Provides interpretability — attention weights show which input tokens the model focuses on
- Basis of **Transformers (BERT, GPT)** — self-attention replaces RNNs entirely

# UNIT 11: APPLICATIONS OF RNNs

| Domain | Application | RNN Type Used |
|---|---|---|
| 🗣️ NLP | **Language Modeling (predict next word)** | Many-to-Many RNN/LSTM |
| 🗣️ NLP | **Machine Translation** | Encoder-Decoder LSTM + Attention |
| 🗣️ NLP | **Sentiment Analysis** | Many-to-One BiLSTM |
| 🗣️ NLP | **Named Entity Recognition** | Many-to-Many BiLSTM |
| 📈 Time-Series | **Stock Price Prediction** | Many-to-One LSTM |
| 📈 Time-Series | **Weather Forecasting** | Many-to-Many LSTM |
| 📈 Time-Series | **Anomaly Detection** | Autoencoder LSTM |
| 🎙️ Speech | **Speech Recognition** | Deep BiRNN / CTC Loss |
| 🎙️ Speech | **Text-to-Speech Synthesis** | Seq2Seq + Attention |
| 🎵 Music | **Music Generation** | Many-to-Many LSTM |

# ⚡ QUICK REVISION SUMMARY

## KEY FORMULAS TO REMEMBER

- **RNN Hidden State:** $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$
- **RNN Output:** $y_t = W_{hy} \cdot h_t + b_y$
- **LSTM Forget:** $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- **LSTM Input:** $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- **LSTM Output Gate:** $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
- **LSTM Cell:** $C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C \cdot [h_{t-1}, x_t])$
- **LSTM Hidden:** $h_t = o_t \odot \tanh(C_t)$
- **GRU Update:** $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$
- **GRU Reset:** $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$
- **Attention:** $\alpha = \text{softmax}(e); \quad c = \Sigma\ \alpha_i h_i$

## TOP 10 EXAM POINTS

- **RNN shares weights across all time steps**
- **Vanishing gradient: use LSTM/GRU; Exploding: use gradient clipping**
- **LSTM has 3 gates + cell state; GRU has 2 gates, no cell state**
- **BiRNN output = 2 × hidden_size (concatenation)**
- **BPTT = unroll RNN + apply backprop + sum gradients over T**
- **Truncated BPTT: backprop through only k steps (faster)**
- **Attention: softmax over scores → weighted sum of encoder outputs**
- **Many-to-Many: Machine translation; Many-to-One: Sentiment analysis**
- **Deep RNN: stack layers; output_l → input_{l+1}**
- **GRU ≈ LSTM but faster (fewer params); LSTM better for very long sequences**